```
%config Completer.use_jedi = False

%matplotlib inline
!pip show tensorflow
!wget -cq https://ti.arc.nasa.gov/c/5 -O naza.zip
!unzip -qqo naza.zip -d battery_data
```

```
Name: tensorflow
Version: 2.10.0
Summary: TensorFlow is an open source machine learning framework for
everyone.
Home-page: https://www.tensorflow.org/
Author: Google Inc.
Author-email: packages@tensorflow.org
License: Apache 2.0
Location: c:\users\kamal\anaconda3\envs\ncmapss_gpu\lib\site-packages
Requires: absl-py, astunparse, flatbuffers, gast, google-pasta,
grpcio, h5py, keras, keras-preprocessing, libclang, numpy, opt-einsum,
packaging, protobuf, setuptools, six, tensorboard, tensorflow-
estimator, tensorflow-io-gcs-filesystem, termcolor, typing-extensions,
wrapt
Required-by:

'wget' is not recognized as an internal or external command,
operable program or batch file.
'unzip' is not recognized as an internal or external command,
operable program or batch file.
```

```
import datetime
import numpy as np
import pandas as pd
from scipy.io import loadmat
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
from sklearn import metrics
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.io import loadmat
```

For the Deep Learning model proposed in [1] it is only necessary to collect the data related to the discharge of the battery, for this a function is created in Python that is in charge of reading this data from the ".mat" file and storing it in memory in two pandas DataFrame for later access. After loading the dataset, a description of the data is made using panda functions to verify if the data loading was correct.

```
def load_data(battery):
  mat =mat =
loadmat('/Users/Kamal/Documents/AUC/thesiswork/Nasa_battery_dataset/'+
battery + '.mat')
  print('Total data in dataset: ', len(mat[battery][0, 0]['cycle']
```

```
[0]))
  counter = 0
  dataset = []
  capacity_data = []

  for i in range(len(mat[battery][0, 0]['cycle'][0])):
    row = mat[battery][0, 0]['cycle'][0, i]
    if row['type'][0] == 'discharge':
      ambient_temperature = row['ambient_temperature'][0][0]
      date_time = datetime.datetime(int(row['time'][0][0]),
                                    int(row['time'][0][1]),
                                    int(row['time'][0][2]),
                                    int(row['time'][0][3]),
                                    int(row['time'][0][4])) +
datetime.timedelta(seconds=int(row['time'][0][5]))
      data = row['data']
      capacity = data[0][0]['Capacity'][0][0]
      for j in range(len(data[0][0]['Voltage_measured'][0])):
        voltage_measured = data[0][0]['Voltage_measured'][0][j]
        current_measured = data[0][0]['Current_measured'][0][j]
        temperature_measured = data[0][0]['Temperature_measured'][0]
[j]
        current_load = data[0][0]['Current_load'][0][j]
        voltage_load = data[0][0]['Voltage_load'][0][j]
        time = data[0][0]['Time'][0][j]
        dataset.append([counter + 1, ambient_temperature, date_time,
capacity,
                        voltage_measured, current_measured,
                        temperature_measured, current_load,
                        voltage_load, time])
      capacity_data.append([counter + 1, ambient_temperature,
date_time, capacity])
      counter = counter + 1
  print(dataset[0])
  return [pd.DataFrame(data=dataset,
                       columns=['cycle', 'ambient_temperature',
'datetime',
                                'capacity', 'voltage_measured',
                                'current_measured',
'temperature_measured',
                                'current_load', 'voltage_load',
'time']),
          pd.DataFrame(data=capacity_data,
                       columns=['cycle', 'ambient_temperature',
'datetime',
                                'capacity'])]
dataset, capacity = load_data('B0005')
pd.set_option('display.max_columns', 10)
```

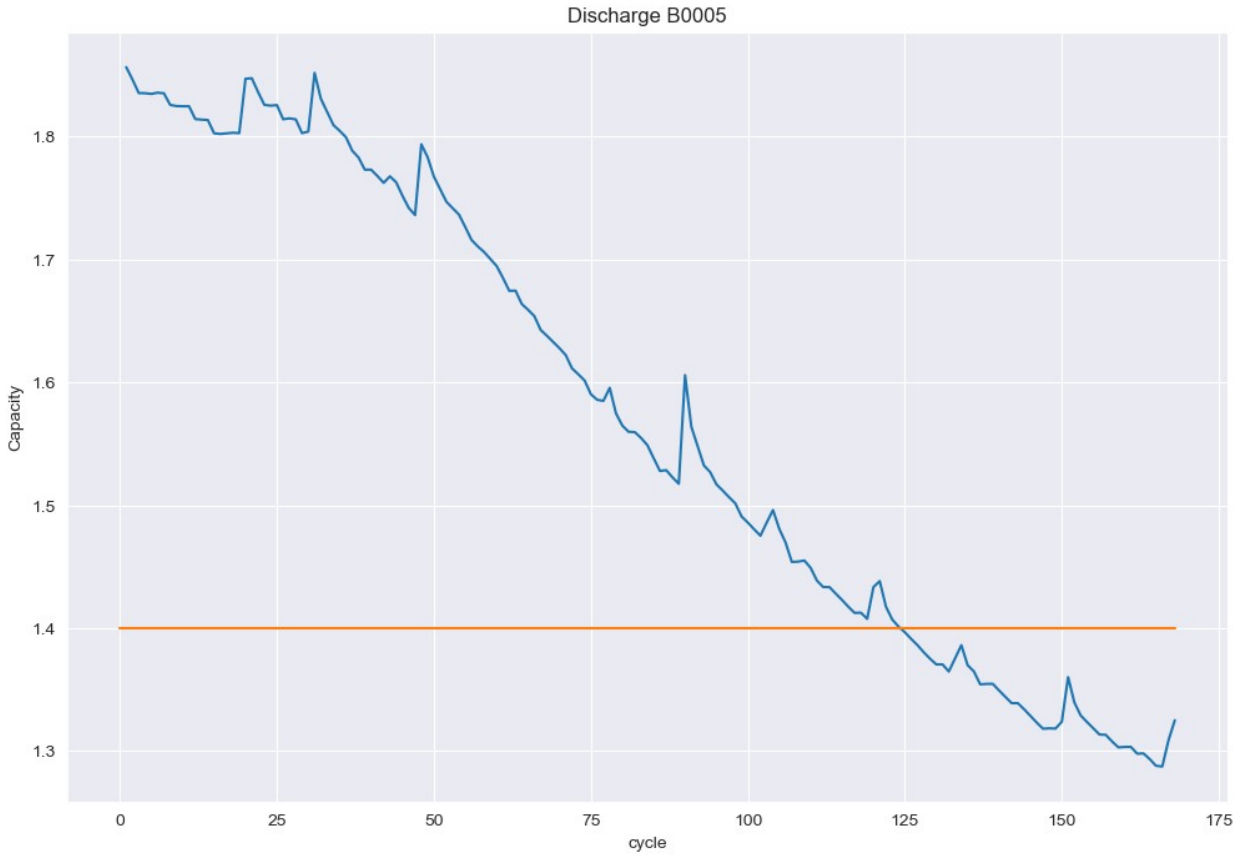
```
print(dataset.head())
dataset.describe()

Total data in dataset:  616
[1, 24, datetime.datetime(2008, 4, 2, 15, 25, 41), 1.8564874208181574,
4.191491807505295, -0.004901589207462691, 24.330033885570543, -0.0006,
0.0, 0.0]
   cycle  ambient_temperature            datetime  capacity
voltage_measured  \
0      1                   24 2008-04-02 15:25:41  1.856487
4.191492
1      1                   24 2008-04-02 15:25:41  1.856487
4.190749
2      1                   24 2008-04-02 15:25:41  1.856487
3.974871
3      1                   24 2008-04-02 15:25:41  1.856487
3.951717
4      1                   24 2008-04-02 15:25:41  1.856487
3.934352

   current_measured  temperature_measured  current_load  voltage_load
time
0         -0.004902             24.330034       -0.0006         0.000
0.000
1         -0.001478             24.325993       -0.0006         4.206
16.781
2         -2.012528             24.389085       -1.9982         3.062
35.703
3         -2.013979             24.544752       -1.9982         3.030
53.781
4         -2.011144             24.731385       -1.9982         3.011
71.922

              cycle  ambient_temperature      capacity
voltage_measured  \
count  50285.000000              50285.0  50285.000000
50285.000000
mean      88.125942                 24.0      1.560345
3.515268
std       45.699687                  0.0      0.182380
0.231778
min        1.000000                 24.0      1.287453
2.455679
25%       50.000000                 24.0      1.386229
3.399384
50%       88.000000                 24.0      1.538237
3.511664
75%      127.000000                 24.0      1.746871
3.660903
max      168.000000                 24.0      1.856487
```

```
4.222920

       current_measured  temperature_measured  current_load
voltage_load  \
count      50285.000000          50285.000000  50285.000000
50285.000000
mean          -1.806032             32.816991      1.362700
2.308406
std            0.610502              3.987515      1.313698
0.800300
min           -2.029098             23.214802     -1.998400
0.000000
25%           -2.013415             30.019392      1.998000
2.388000
50%           -2.012312             32.828944      1.998200
2.533000
75%           -2.011052             35.920887      1.998200
2.690000
max            0.007496             41.450232      1.998400
4.238000

                time
count  50285.000000
mean    1546.208924
std      906.640295
min        0.000000
25%      768.563000
50%     1537.031000
75%     2305.984000
max     3690.234000
```

The following graph shows the aging process of the battery as the charge cycles progress. The horizontal line represents the threshold related to what can be considered the end of the battery's life cycle.

```
plot_df = capacity.loc[(capacity['cycle']>=1),['cycle','capacity']]
sns.set_style("darkgrid")
plt.figure(figsize=(12, 8))
plt.plot(plot_df['cycle'], plot_df['capacity'])
#Draw threshold
plt.plot([0.,len(capacity)], [1.4, 1.4])
plt.ylabel('Capacity')
# make x-axis ticks legible
adf = plt.gca().get_xaxis().get_major_formatter()
plt.xlabel('cycle')
plt.title('Discharge B0005')
```

```
Text(0.5, 1.0, 'Discharge B0005')
```




It is also necessary to calculate the SoH of the battery, since this is the data that will be predicted using the * deep learning * model.

```
attrib=['cycle', 'datetime', 'capacity']
dis_ele = capacity[attrib]
C = dis_ele['capacity'][0]
for i in range(len(dis_ele)):
    dis_ele['SoH']=(dis_ele['capacity'])/C
print(dis_ele.head(5))
```

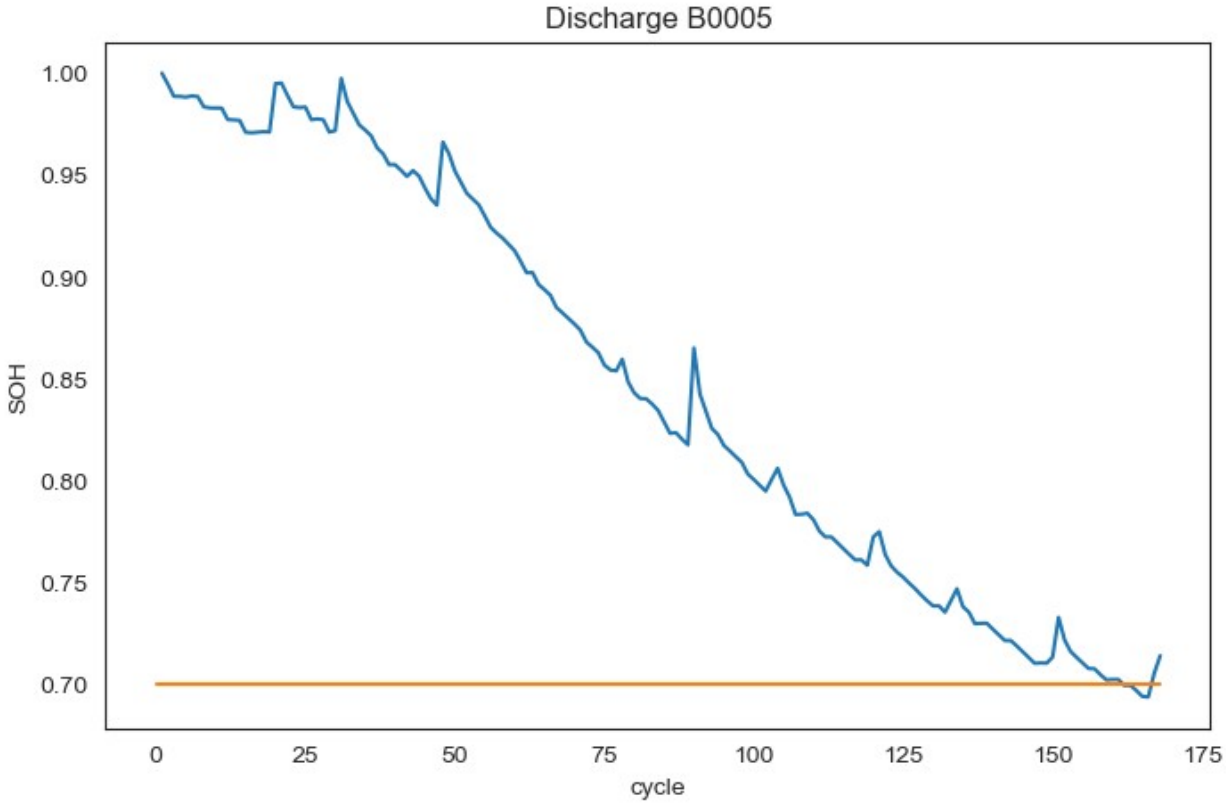
```
   cycle            datetime  capacity       SoH
0      1 2008-04-02 15:25:41  1.856487  1.000000
1      2 2008-04-02 19:43:48  1.846327  0.994527
2      3 2008-04-03 00:01:06  1.835349  0.988614
3      4 2008-04-03 04:16:37  1.835263  0.988567
4      5 2008-04-03 08:33:25  1.834646  0.988235
```

Similarly to what has been done previously, a graph of the SoH is made for each cycle, the horizontal line represents the threshold of 70% in which the battery already fulfills its life cycle and it is advisable to make the change.

```
plot_df = dis_ele.loc[(dis_ele['cycle']>=1),['cycle','SoH']]
sns.set_style("white")
```

```
plt.figure(figsize=(8, 5))
plt.plot(plot_df['cycle'], plot_df['SoH'])
#Draw threshold
plt.plot([0.,len(capacity)], [0.70, 0.70])
plt.ylabel('SOH')
# make x-axis ticks legible
adf = plt.gca().get_xaxis().get_major_formatter()
plt.xlabel('cycle')
plt.title('Discharge B0005')
```

```
Text(0.5, 1.0, 'Discharge B0005')
```




```
C = dataset['capacity'][0]
soh = []
for i in range(len(dataset)):
  soh.append([dataset['capacity'][i] / C])
soh = pd.DataFrame(data=soh, columns=['SoH'])

attribs=['capacity', 'voltage_measured', 'current_measured',
         'temperature_measured', 'current_load', 'voltage_load',
'time']
train_dataset = dataset[attribs]
sc = MinMaxScaler(feature_range=(0,1))
train_dataset = sc.fit_transform(train_dataset)
```

```
print(train_dataset.shape)
print(soh.shape)
```

```
(50285, 7)
(50285, 1)
```

Preparation of the model, 3 dense layers are used, and the parameters are used as they are in the paper: 3 dense layers and one dropout, and one of the ADAM type is used as optimizer

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import LSTM
from tensorflow.keras.optimizers import Adam

model = Sequential()
model.add(Dense(8, activation='relu',
input_dim=train_dataset.shape[1]))
model.add(Dense(8, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dropout(rate=0.25))
model.add(Dense(1))
model.summary()
model.compile(optimizer=Adam(beta_1=0.9, beta_2=0.999, epsilon=1e-08),
loss='mean_absolute_error')
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense (Dense)               (None, 8)                 64

 dense_1 (Dense)             (None, 8)                 72

 dense_2 (Dense)             (None, 8)                 72

 dropout (Dropout)           (None, 8)                 0

 dense_3 (Dense)             (None, 1)                 9

=================================================================
Total params: 217
Trainable params: 217
Non-trainable params: 0
_________________________________________________________________
```

## The model is trained, 50 epochs are used for training

```
model.fit(x=train_dataset, y=soh.to_numpy(), batch_size=25, epochs=30)

Epoch 1/30
2012/2012 [==============================] - 2s 761us/step - loss:
0.0997
Epoch 2/30
2012/2012 [==============================] - 1s 664us/step - loss:
0.0230
Epoch 3/30
2012/2012 [==============================] - 1s 670us/step - loss:
0.0230
Epoch 4/30
2012/2012 [==============================] - 1s 698us/step - loss:
0.0226
Epoch 5/30
2012/2012 [==============================] - 2s 766us/step - loss:
0.0221
Epoch 6/30
2012/2012 [==============================] - 2s 750us/step - loss:
0.0224
Epoch 7/30
2012/2012 [==============================] - 2s 812us/step - loss:
0.0220
Epoch 8/30
2012/2012 [==============================] - 1s 667us/step - loss:
0.0222
Epoch 9/30
2012/2012 [==============================] - 1s 672us/step - loss:
0.0223
Epoch 10/30
2012/2012 [==============================] - 1s 683us/step - loss:
0.0223
Epoch 11/30
2012/2012 [==============================] - 1s 739us/step - loss:
0.0222
Epoch 12/30
2012/2012 [==============================] - 1s 713us/step - loss:
0.0221
Epoch 13/30
2012/2012 [==============================] - 2s 749us/step - loss:
0.0221
Epoch 14/30
2012/2012 [==============================] - 1s 733us/step - loss:
0.0222
Epoch 15/30
2012/2012 [==============================] - 1s 672us/step - loss:
0.0223
Epoch 16/30
```

```
2012/2012 [==============================] - 1s 742us/step - loss:
0.0219
Epoch 17/30
2012/2012 [==============================] - 1s 713us/step - loss:
0.0219
Epoch 18/30
2012/2012 [==============================] - 1s 693us/step - loss:
0.0221
Epoch 19/30
2012/2012 [==============================] - 1s 701us/step - loss:
0.0219
Epoch 20/30
2012/2012 [==============================] - 1s 713us/step - loss:
0.0220
Epoch 21/30
2012/2012 [==============================] - 1s 695us/step - loss:
0.0221
Epoch 22/30
2012/2012 [==============================] - 2s 781us/step - loss:
0.0221
Epoch 23/30
2012/2012 [==============================] - 1s 693us/step - loss:
0.0220
Epoch 24/30
2012/2012 [==============================] - 1s 685us/step - loss:
0.0219
Epoch 25/30
2012/2012 [==============================] - 2s 748us/step - loss:
0.0223
Epoch 26/30
2012/2012 [==============================] - 1s 742us/step - loss:
0.0221
Epoch 27/30
2012/2012 [==============================] - 1s 737us/step - loss:
0.0223
Epoch 28/30
2012/2012 [==============================] - 2s 751us/step - loss:
0.0221
Epoch 29/30
2012/2012 [==============================] - 1s 682us/step - loss:
0.0222
Epoch 30/30
2012/2012 [==============================] - 1s 742us/step - loss:
0.0224

<keras.callbacks.History at 0x246eb73bf70>
```

```
dataset_val, capacity_val = load_data('B0006')
attrib=['cycle', 'datetime', 'capacity']
dis_ele = capacity_val[attrib]
```

```
C = dis_ele['capacity'][0]
for i in range(len(dis_ele)):
    dis_ele['SoH']=(dis_ele['capacity']) / C
print(dataset_val.head(5))
print(dis_ele.head(5))
```

```
Total data in dataset:  616
[1, 24, datetime.datetime(2008, 4, 2, 15, 25, 41), 2.035337591005598,
4.179799607333447, -0.0023663271409738672, 24.277567510331888, -
0.0006, 0.0, 0.0]
   cycle  ambient_temperature            datetime  capacity
voltage_measured  \
0      1                   24 2008-04-02 15:25:41  2.035338
4.179800
1      1                   24 2008-04-02 15:25:41  2.035338
4.179823
2      1                   24 2008-04-02 15:25:41  2.035338
3.966528
3      1                   24 2008-04-02 15:25:41  2.035338
3.945886
4      1                   24 2008-04-02 15:25:41  2.035338
3.930354

   current_measured  temperature_measured  current_load  voltage_load
time
0         -0.002366             24.277568       -0.0006         0.000
0.000
1          0.000434             24.277073       -0.0006         4.195
16.781
2         -2.014242             24.366226       -1.9990         3.070
35.703
3         -2.008730             24.515123       -1.9990         3.045
53.781
4         -2.013381             24.676053       -1.9990         3.026
71.922
   cycle            datetime  capacity       SoH
0      1 2008-04-02 15:25:41  2.035338  1.000000
1      2 2008-04-02 19:43:48  2.025140  0.994990
2      3 2008-04-03 00:01:06  2.013326  0.989185
3      4 2008-04-03 04:16:37  2.013285  0.989165
4      5 2008-04-03 08:33:25  2.000528  0.982898
```

A table is created containing the real SoH and the SoH predicted by the network and the root of the mean square error is calculated.

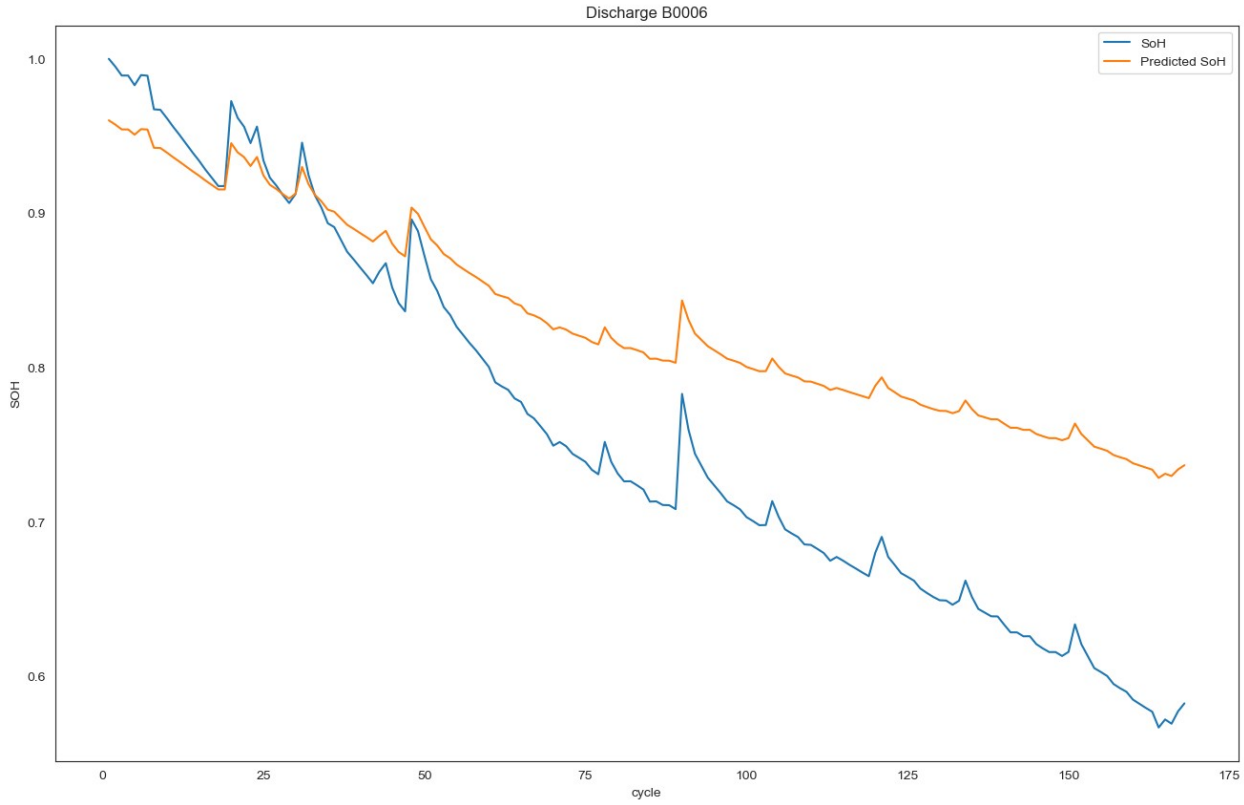
```
attrib=['capacity', 'voltage_measured', 'current_measured',
        'temperature_measured', 'current_load', 'voltage_load',
'time']
soh_pred = model.predict(sc.fit_transform(dataset_val[attrib]))
```

```
print(soh_pred.shape)

C = dataset_val['capacity'][0]
soh = []
for i in range(len(dataset_val)):
  soh.append(dataset_val['capacity'][i] / C)
new_soh = dataset_val.loc[(dataset_val['cycle'] >= 1), ['cycle']]
new_soh['SoH'] =  soh
new_soh['NewSoH'] = soh_pred
new_soh = new_soh.groupby(['cycle']).mean().reset_index()
print(new_soh.head(10))
rms = np.sqrt(mean_squared_error(new_soh['SoH'], new_soh['NewSoH']))
print('Root Mean Square Error: ', rms)
```

```
1572/1572 [==============================] - 1s 536us/step
(50285, 1)
   cycle       SoH    NewSoH
0      1  1.000000  0.960161
1      2  0.994990  0.957421
2      3  0.989185  0.954270
3      4  0.989165  0.954264
4      5  0.982898  0.950870
5      6  0.989467  0.954442
6      7  0.989075  0.954222
7      8  0.967304  0.942388
8      9  0.966997  0.942226
9     10  0.961625  0.939314
Root Mean Square Error:  0.09140341612360732
```

```
plot_df = new_soh.loc[(new_soh['cycle']>=1),['cycle','SoH', 'NewSoH']]
sns.set_style("white")
plt.figure(figsize=(16, 10))
plt.plot(plot_df['cycle'], plot_df['SoH'], label='SoH')
plt.plot(plot_df['cycle'], plot_df['NewSoH'], label='Predicted SoH')
#Draw threshold
#plt.plot([0.,len(capacity)], [0.70, 0.70], label='Threshold')
plt.ylabel('SOH')
# make x-axis ticks legible
adf = plt.gca().get_xaxis().get_major_formatter()
plt.xlabel('cycle')
plt.legend()
plt.title('Discharge B0006')
```

```
Text(0.5, 1.0, 'Discharge B0006')
```




```
dataset_val, capacity_val = load_data('B0005')
attrib=['cycle', 'datetime', 'capacity']
dis_ele = capacity_val[attrib]
rows=['cycle','capacity']
dataset=dis_ele[rows]
data_train=dataset[(dataset['cycle']<50)]
data_set_train=data_train.iloc[:,1:2].values
data_test=dataset[(dataset['cycle']>=50)]
data_set_test=data_test.iloc[:,1:2].values

sc=MinMaxScaler(feature_range=(0,1))
data_set_train=sc.fit_transform(data_set_train)
data_set_test=sc.transform(data_set_test)

X_train=[]
y_train=[]
#take the last 10t to predict 10t+1
for i in range(10,49):
    X_train.append(data_set_train[i-10:i,0])
    y_train.append(data_set_train[i,0])
X_train,y_train=np.array(X_train),np.array(y_train)

X_train=np.reshape(X_train,(X_train.shape[0],X_train.shape[1],1))
```

```
Total data in dataset:  616
[1, 24, datetime.datetime(2008, 4, 2, 15, 25, 41), 1.8564874208181574,
4.191491807505295, -0.004901589207462691, 24.330033885570543, -0.0006,
0.0, 0.0]

import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout, Masking,
TimeDistributed
from keras.models import Sequential
from keras.layers import Dense, Conv1D
from keras.layers import Dropout, Input
from keras import initializers
from keras.layers import MaxPooling1D
from keras.models import Model
from keras.layers import TimeDistributed, Flatten
from keras.layers.core import Dense, Activation, Dropout
from keras.callbacks import EarlyStopping
from keras.callbacks import ModelCheckpoint

input_shape=(X_train.shape[1],1)

from sklearn.model_selection import train_test_split

# Assuming `X_train` and `y_train` are your full training datasets
# Splitting the dataset into training (80%) and validation (20%) sets
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train,
test_size=0.2, random_state=42)


from tensorflow import keras
from keras_tuner import Hyperband

def build_model(hp):
    model = keras.Sequential([
        keras.layers.LSTM(
            units=hp.Int('units', min_value=32, max_value=512,
step=32),
            input_shape=[X_train.shape[1], X_train.shape[2]]),  #
Example input shape
        keras.layers.Dense(1, activation='linear')
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(
            hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])),
        loss='mean_absolute_error',
        metrics=['mean_absolute_error']
    )
```

```
    return model

# Now, setting up the Hyperband tuner
tuner = Hyperband(
    build_model,
    objective='val_mean_absolute_error',
    max_epochs=10,  # Adjust based on your dataset size and model
complexity
    directory='hyperband',
    project_name='battery_health_optimization'
)

# Assuming X_train, y_train are your training data
# Split your data into training and validation here, if not already
done

# Start searching for the best model hyperparameters
tuner.search(X_train, y_train, epochs=10, validation_split=0.2)  #
Adjust epochs, validation_split as needed

# After search is complete, you can get the best model like this:
best_model = tuner.get_best_models(num_models=1)[0]
best_model.summary()
```

```
Trial 30 Complete [00h 00m 03s]
val_mean_absolute_error: 0.15336330235004425

Best val_mean_absolute_error So Far: 0.10463542491197586
Total elapsed time: 00h 01m 10s
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm (LSTM)                 (None, 352)               498432

 dense (Dense)               (None, 1)                 353

=================================================================
Total params: 498,785
Trainable params: 498,785
Non-trainable params: 0
_________________________________________________________________
```

```
best_model.fit(X_train,y_train,epochs=100,batch_size=25)
```

```
Epoch 1/100
1/2 [==============>..............] - ETA: 0s - loss: 0.0255 -
mean_absolute_error: 0.0255

2/2 [==============================] - 0s 15ms/step - loss: 0.0273 -
mean_absolute_error: 0.0273
```

```
Epoch 2/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0254 -
mean_absolute_error: 0.0254
Epoch 3/100
2/2 [==============================] - 0s 17ms/step - loss: 0.0257 -
mean_absolute_error: 0.0257
Epoch 4/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0303 -
mean_absolute_error: 0.0303
Epoch 5/100
2/2 [==============================] - 0s 13ms/step - loss: 0.0412 -
mean_absolute_error: 0.0412
Epoch 6/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0413 -
mean_absolute_error: 0.0413
Epoch 7/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0353 -
mean_absolute_error: 0.0353
Epoch 8/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0321 -
mean_absolute_error: 0.0321
Epoch 9/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0268 -
mean_absolute_error: 0.0268
Epoch 10/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0313 -
mean_absolute_error: 0.0313
Epoch 11/100
2/2 [==============================] - 0s 12ms/step - loss: 0.0301 -
mean_absolute_error: 0.0301
Epoch 12/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0255 -
mean_absolute_error: 0.0255
Epoch 13/100
2/2 [==============================] - 0s 18ms/step - loss: 0.0290 -
mean_absolute_error: 0.0290
Epoch 14/100
2/2 [==============================] - 0s 17ms/step - loss: 0.0274 -
mean_absolute_error: 0.0274
Epoch 15/100
2/2 [==============================] - 0s 13ms/step - loss: 0.0314 -
mean_absolute_error: 0.0314
Epoch 16/100
2/2 [==============================] - 0s 17ms/step - loss: 0.0362 -
mean_absolute_error: 0.0362
Epoch 17/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0247 -
mean_absolute_error: 0.0247
Epoch 18/100
```

```
2/2 [==============================] - 0s 14ms/step - loss: 0.0263 -
mean_absolute_error: 0.0263
Epoch 19/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0293 -
mean_absolute_error: 0.0293
Epoch 20/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0314 -
mean_absolute_error: 0.0314
Epoch 21/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0322 -
mean_absolute_error: 0.0322
Epoch 22/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0238 -
mean_absolute_error: 0.0238
Epoch 23/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0193 -
mean_absolute_error: 0.0193
Epoch 24/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0180 -
mean_absolute_error: 0.0180
Epoch 25/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0221 -
mean_absolute_error: 0.0221
Epoch 26/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0258 -
mean_absolute_error: 0.0258
Epoch 27/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0209 -
mean_absolute_error: 0.0209
Epoch 28/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0190 -
mean_absolute_error: 0.0190
Epoch 29/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0209 -
mean_absolute_error: 0.0209
Epoch 30/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0259 -
mean_absolute_error: 0.0259
Epoch 31/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0269 -
mean_absolute_error: 0.0269
Epoch 32/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0233 -
mean_absolute_error: 0.0233
Epoch 33/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0202 -
mean_absolute_error: 0.0202
Epoch 34/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0123 -
```

```
mean_absolute_error: 0.0123
Epoch 35/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0242 -
mean_absolute_error: 0.0242
Epoch 36/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0224 -
mean_absolute_error: 0.0224
Epoch 37/100
2/2 [==============================] - 0s 13ms/step - loss: 0.0296 -
mean_absolute_error: 0.0296
Epoch 38/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0345 -
mean_absolute_error: 0.0345
Epoch 39/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0300 -
mean_absolute_error: 0.0300
Epoch 40/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0267 -
mean_absolute_error: 0.0267
Epoch 41/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0234 -
mean_absolute_error: 0.0234
Epoch 42/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0209 -
mean_absolute_error: 0.0209
Epoch 43/100
2/2 [==============================] - 0s 13ms/step - loss: 0.0189 -
mean_absolute_error: 0.0189
Epoch 44/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0189 -
mean_absolute_error: 0.0189
Epoch 45/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0173 -
mean_absolute_error: 0.0173
Epoch 46/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0162 -
mean_absolute_error: 0.0162
Epoch 47/100
2/2 [==============================] - 0s 13ms/step - loss: 0.0159 -
mean_absolute_error: 0.0159
Epoch 48/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0223 -
mean_absolute_error: 0.0223
Epoch 49/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0198 -
mean_absolute_error: 0.0198
Epoch 50/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0294 -
mean_absolute_error: 0.0294
```

```
Epoch 51/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0311 -
mean_absolute_error: 0.0311
Epoch 52/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0219 -
mean_absolute_error: 0.0219
Epoch 53/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0282 -
mean_absolute_error: 0.0282
Epoch 54/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0235 -
mean_absolute_error: 0.0235
Epoch 55/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0259 -
mean_absolute_error: 0.0259
Epoch 56/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0292 -
mean_absolute_error: 0.0292
Epoch 57/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0211 -
mean_absolute_error: 0.0211
Epoch 58/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0217 -
mean_absolute_error: 0.0217
Epoch 59/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0189 -
mean_absolute_error: 0.0189
Epoch 60/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0225 -
mean_absolute_error: 0.0225
Epoch 61/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0205 -
mean_absolute_error: 0.0205
Epoch 62/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0216 -
mean_absolute_error: 0.0216
Epoch 63/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0198 -
mean_absolute_error: 0.0198
Epoch 64/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0216 -
mean_absolute_error: 0.0216
Epoch 65/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0227 -
mean_absolute_error: 0.0227
Epoch 66/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0271 -
mean_absolute_error: 0.0271
Epoch 67/100
```

```
2/2 [==============================] - 0s 15ms/step - loss: 0.0196 -
mean_absolute_error: 0.0196
Epoch 68/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0268 -
mean_absolute_error: 0.0268
Epoch 69/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0263 -
mean_absolute_error: 0.0263
Epoch 70/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0193 -
mean_absolute_error: 0.0193
Epoch 71/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0177 -
mean_absolute_error: 0.0177
Epoch 72/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0222 -
mean_absolute_error: 0.0222
Epoch 73/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0212 -
mean_absolute_error: 0.0212
Epoch 74/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0328 -
mean_absolute_error: 0.0328
Epoch 75/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0280 -
mean_absolute_error: 0.0280
Epoch 76/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0214 -
mean_absolute_error: 0.0214
Epoch 77/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0277 -
mean_absolute_error: 0.0277
Epoch 78/100
2/2 [==============================] - 0s 24ms/step - loss: 0.0321 -
mean_absolute_error: 0.0321
Epoch 79/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0284 -
mean_absolute_error: 0.0284
Epoch 80/100
2/2 [==============================] - 0s 13ms/step - loss: 0.0209 -
mean_absolute_error: 0.0209
Epoch 81/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0212 -
mean_absolute_error: 0.0212
Epoch 82/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0216 -
mean_absolute_error: 0.0216
Epoch 83/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0205 -
```

```
mean_absolute_error: 0.0205
Epoch 84/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0200 -
mean_absolute_error: 0.0200
Epoch 85/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0162 -
mean_absolute_error: 0.0162
Epoch 86/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0166 -
mean_absolute_error: 0.0166
Epoch 87/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0153 -
mean_absolute_error: 0.0153
Epoch 88/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0201 -
mean_absolute_error: 0.0201
Epoch 89/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0187 -
mean_absolute_error: 0.0187
Epoch 90/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0251 -
mean_absolute_error: 0.0251
Epoch 91/100
2/2 [==============================] - 0s 13ms/step - loss: 0.0216 -
mean_absolute_error: 0.0216
Epoch 92/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0180 -
mean_absolute_error: 0.0180
Epoch 93/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0161 -
mean_absolute_error: 0.0161
Epoch 94/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0209 -
mean_absolute_error: 0.0209
Epoch 95/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0209 -
mean_absolute_error: 0.0209
Epoch 96/100
2/2 [==============================] - 0s 14ms/step - loss: 0.0142 -
mean_absolute_error: 0.0142
Epoch 97/100
2/2 [==============================] - 0s 16ms/step - loss: 0.0166 -
mean_absolute_error: 0.0166
Epoch 98/100
2/2 [==============================] - 0s 18ms/step - loss: 0.0201 -
mean_absolute_error: 0.0201
Epoch 99/100
2/2 [==============================] - 0s 15ms/step - loss: 0.0225 -
mean_absolute_error: 0.0225
Epoch 100/100
```

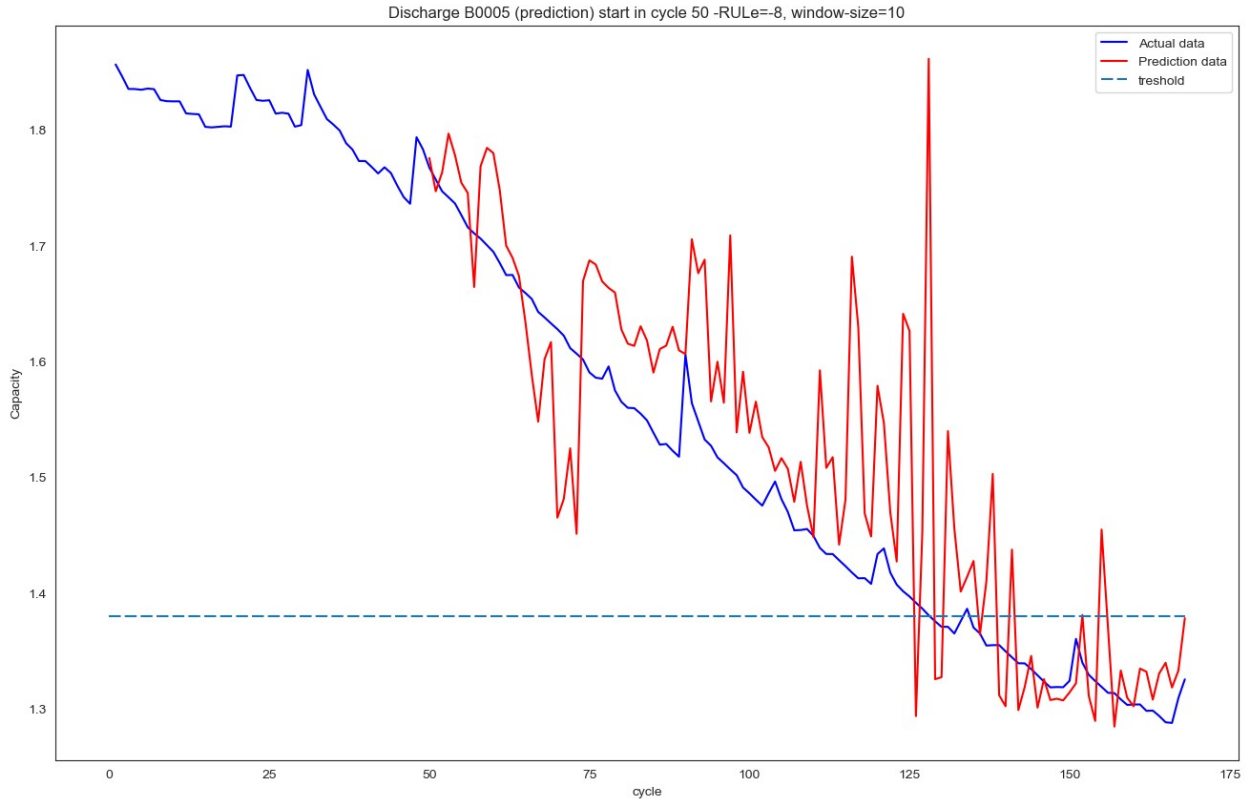
```
2/2 [==============================] - 0s 15ms/step - loss: 0.0211 -
mean_absolute_error: 0.0211

<keras.callbacks.History at 0x246f80cb550>

print(len(data_test))
data_total=pd.concat((data_train['capacity'],
data_test['capacity']),axis=0)
inputs=data_total[len(data_total)-len(data_test)-10:].values
inputs=inputs.reshape(-1,1)
inputs=sc.transform(inputs)

119

X_test=[]
for i in range(10,129):
    X_test.append(inputs[i-10:i,0])
X_test=np.array(X_test)
X_test=np.reshape(X_test,(X_test.shape[0],X_test.shape[1],1))
pred=best_model.predict(X_test)
print(pred.shape)
pred=sc.inverse_transform(pred)
pred=pred[:,0]
tests=data_test.iloc[:,1:2]
rmse = np.sqrt(mean_squared_error(tests, pred))
print('Test RMSE: %.3f' % rmse)
metrics.r2_score(tests,pred)

1/4 [======>.....................] - ETA: 0s

4/4 [==============================] - 0s 9ms/step
(119, 1)
Test RMSE: 0.094

0.5160889383558337
```

As can be seen, the mean RMSE is 0.05 (5%), which is very close to the values observed in the literature using this type of network.

```
ln = len(data_train)
data_test['pre']=pred
plot_df = dataset.loc[(dataset['cycle']>=1),['cycle','capacity']]
plot_per = data_test.loc[(data_test['cycle']>=ln),['cycle','pre']]
plt.figure(figsize=(16, 10))
plt.plot(plot_df['cycle'], plot_df['capacity'], label="Actual data",
color='blue')
plt.plot(plot_per['cycle'],plot_per['pre'],label="Prediction data",
color='red')
#Draw threshold
plt.plot([0.,168], [1.38, 1.38],dashes=[6, 2], label="treshold")
```

```
plt.ylabel('Capacity')
# make x-axis ticks legible
adf = plt.gca().get_xaxis().get_major_formatter()
plt.xlabel('cycle')
plt.legend()
plt.title('Discharge B0005 (prediction) start in cycle 50 -RULe=-8,
window-size=10')
```

```
C:\Users\Kamal\AppData\Local\Temp\ipykernel_35212\1198164500.py:2:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  data_test['pre']=pred

Text(0.5, 1.0, 'Discharge B0005 (prediction) start in cycle 50 -RULe=-
8, window-size=10')
```




Finally, it can be seen in the graph that the capacity value and how it behaves over time is very close to the real value and supporting these data, the error in the estimation of the RUL was -8 which makes us understand that The model went ahead by 8 cycles to estimate that the battery reached its end of life.

```
pred=0
Afil=0
Pfil=0
a=data_test['capacity'].values
b=data_test['pre'].values
j=0
k=0
for i in range(len(a)):
    actual=a[i]

    if actual<=1.38:
        j=i
        Afil=j
        break
for i in range(len(a)):
    pred=b[i]
    if pred< 1.38:
        k=i
        Pfil=k
        break
print("The Actual fail at cycle number: "+ str(Afil+ln))
print("The prediction fail at cycle number: "+ str(Pfil+ln))
RULerror=Pfil-Afil
print("The error of RUL= "+ str(RULerror)+ " Cycle(s)")
```

```
The Actual fail at cycle number: 128
The prediction fail at cycle number: 125
The error of RUL= -3 Cycle(s)
```

```
from keras.models import load_model
import json

# Assuming 'your_model' is your Keras model and 'history' is the
history object returned by model.fit()
model = best_model

# Step 1: Save your Keras model
model.save('Hyperband sequential Optimization.h5')  # Save the model




# Step 3: Load the Keras model
loaded_model = load_model('Hyperband sequential Optimization.h5')
print("Model loaded successfully.")
```

```
WARNING:tensorflow:Error in loading the saved optimizer state. As a result, your model is starting with a freshly initialized optimizer.
Model loaded successfully.
```